

Part 1

EdgeImpulse Public Link: <https://studio.edgeimpulse.com/studio/1003027/impulse/1/deployment>

1. Raw inputs: State the raw inputs used in the Edge Impulse project.

The raw input includes three variables, *accX*, *accY*, and *accZ*.

2. NN classifier features: State the type of features passed into the NN classifier, the number of input features, and at least five actual feature names used to train the classifier.

From raw data we generated 33 input features that would be useful for the NN classifier. Some of the prominent features include *accX RMS*, *accX Peak 1 Height*, *accY Peak 3 Height*, *accX Peak 1 Freq*, *accY RMS*, *accY Peak 2 Freq*, *accY Peak 1 Height*, among others, just to name a few.

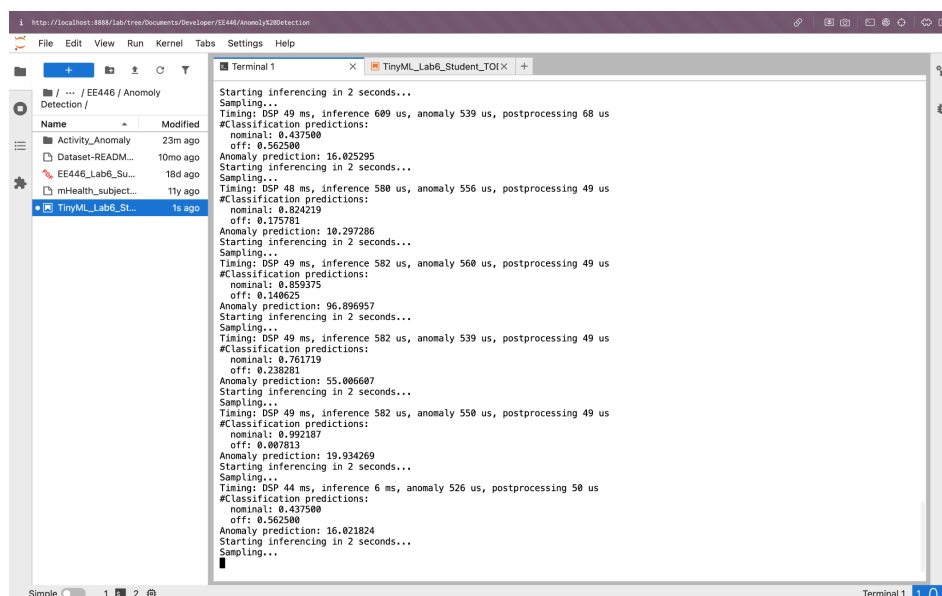
3. NN classifier outputs: State the outputs of the NN classifier.

The output layer of the NN classifier gives two classes, *Nominal* and *Off*.

4. Anomaly detector features: State the type of features passed into the anomaly detector, the number of features used, and the feature names used to train the anomaly detector.

Four engineered features were used in the anomaly detector training: *accX RMS*, *accX Peak 1 Height*, *accY Spectral Power 2.0 - 5.0 Hz*, and *accZ Spectral Power 2.0 - 5.0 Hz*.

5. Deployment evidence: Include a screenshot showing the Edge Impulse model running on the Arduino Nano 33 BLE Sense and producing inference results.



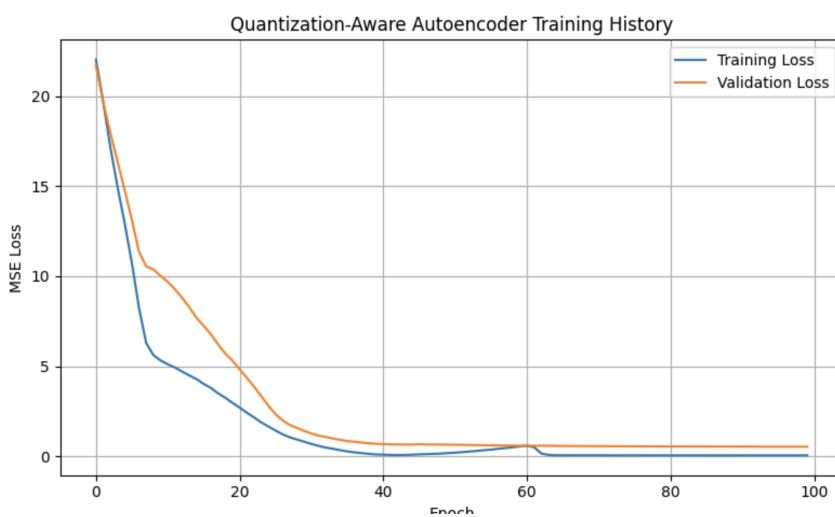
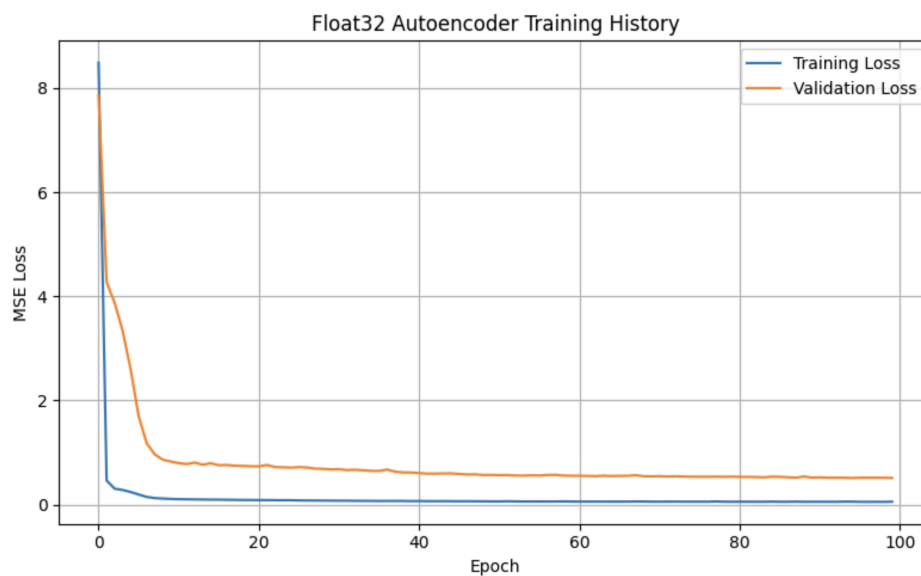
The screenshot shows the Edge Impulse Studio web interface. On the left, a file explorer displays a project named 'EE446 / Anomaly' with several files including 'Activity_Anomaly', 'Dataset-READM...', 'EE446_Lab6_Su...', 'mHealth_subject...', and 'TinyML_Lab6_St...'. The 'TinyML_Lab6_St...' file is selected. On the right, a terminal window titled 'Terminal 1' shows the output of the model's inference process. The output includes timing information for DSP, inference, anomaly detection, and postprocessing, as well as classification predictions for 'nominal' and 'off' classes. The predictions are shown as 'nominal: 0.437500' and 'off: 0.562500' for several samples.

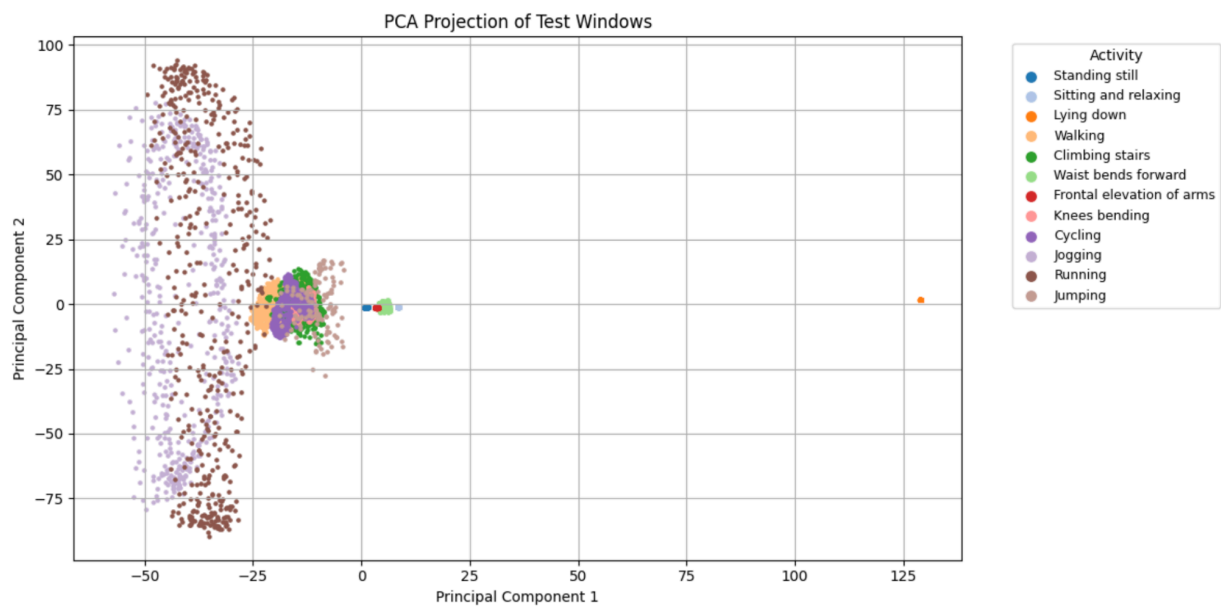
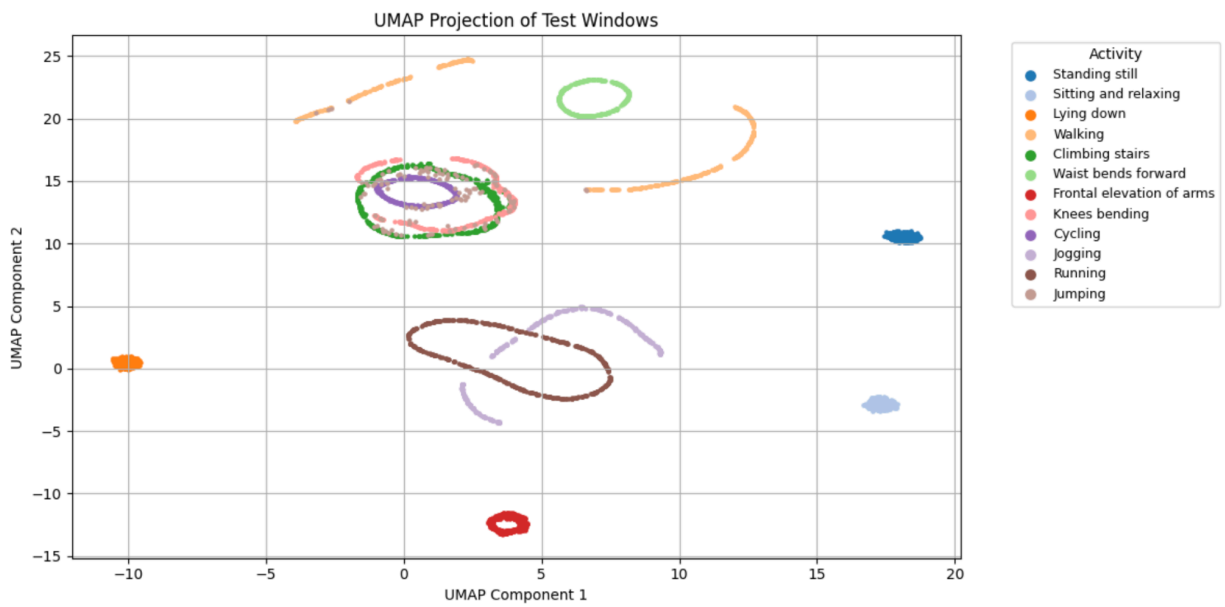
```
Starting inferecing in 2 seconds...
Sampling...
Timing: DSP 49 ms, inference 609 us, anomaly 539 us, postprocessing 68 us
#Classification predictions:
nominal: 0.437500
off: 0.562500
Anomaly prediction: 16.025295
Starting inferecing in 2 seconds...
Sampling...
Timing: DSP 48 ms, inference 588 us, anomaly 556 us, postprocessing 49 us
#Classification predictions:
nominal: 0.824219
off: 0.175781
Anomaly prediction: 10.297286
Starting inferecing in 2 seconds...
Sampling...
Timing: DSP 49 ms, inference 582 us, anomaly 560 us, postprocessing 49 us
#Classification predictions:
nominal: 0.859375
off: 0.140625
Anomaly prediction: 96.896957
Starting inferecing in 2 seconds...
Sampling...
Timing: DSP 49 ms, inference 582 us, anomaly 539 us, postprocessing 49 us
#Classification predictions:
nominal: 0.761719
off: 0.238281
Anomaly prediction: 55.006607
Starting inferecing in 2 seconds...
Sampling...
Timing: DSP 49 ms, inference 582 us, anomaly 550 us, postprocessing 49 us
#Classification predictions:
nominal: 0.992187
off: 0.007813
Anomaly prediction: 19.934269
Starting inferecing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 6 ms, anomaly 526 us, postprocessing 58 us
#Classification predictions:
nominal: 0.437500
off: 0.562500
Anomaly prediction: 16.021824
Starting inferecing in 2 seconds...
Sampling...
```

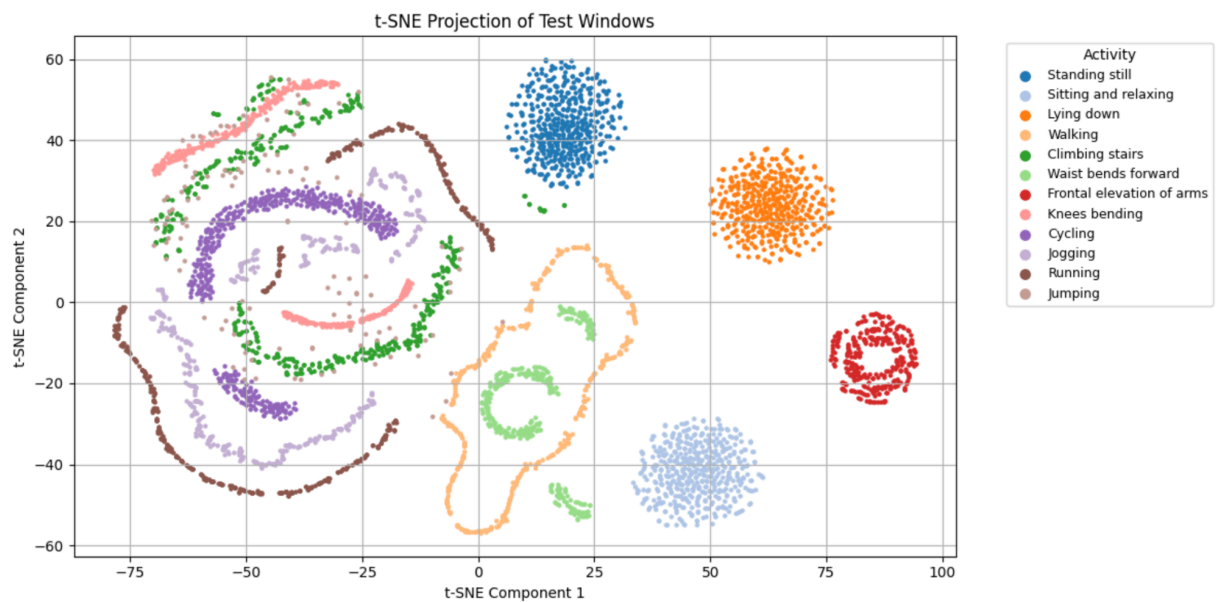
6. Short interpretation: Briefly interpret the observed deployment results. Discuss whether the classifier output should always be trusted and under what conditions it may or may not be reliable.

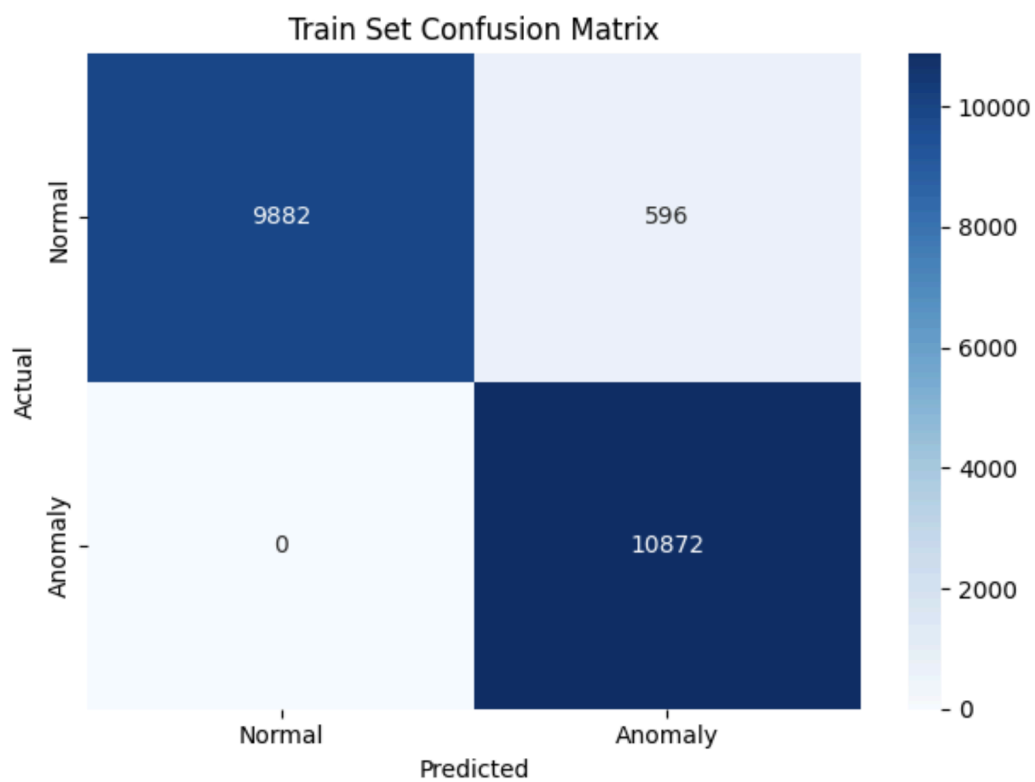
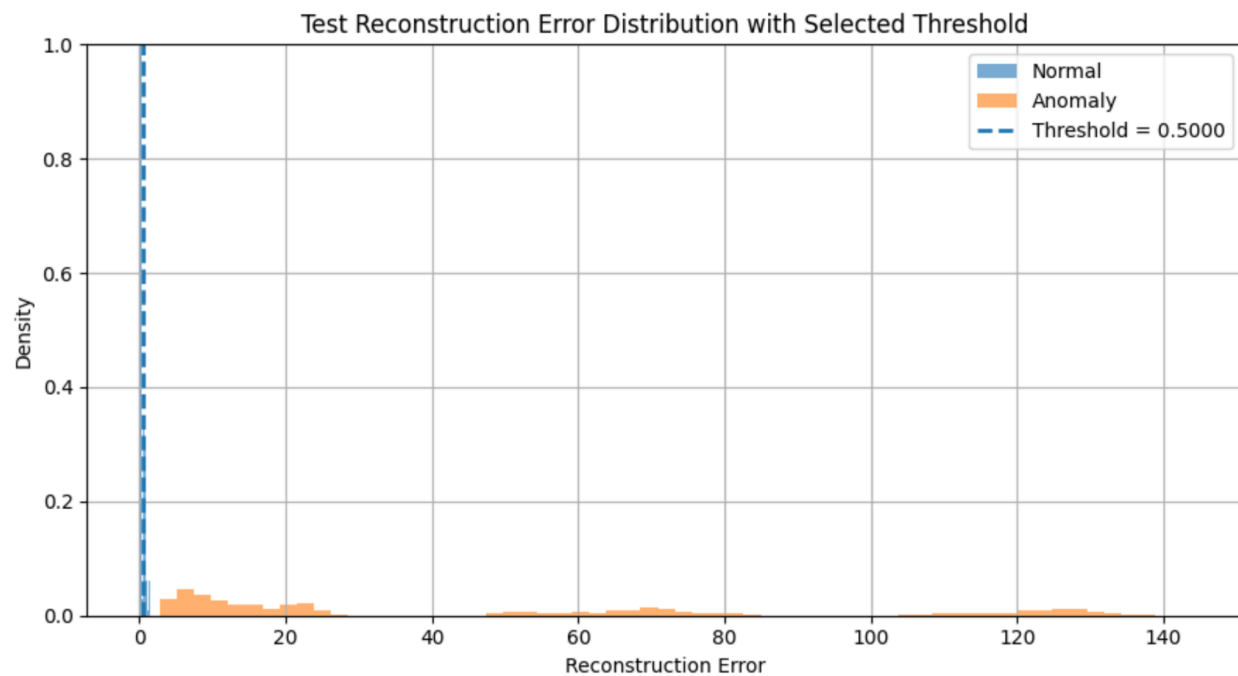
The deployment results show a mixture of trustworthiness and things to look out for. The classifier output is generally trustworthy if the predictions are close to either end of the spectrum (high-likelihood nominal or high-likelihood off) and the anomaly score is low. However, if the confidence level is around the middle close to the class boundary AND the anomaly score is high, then it would be good to exercise caution as we are in uncharted territory.

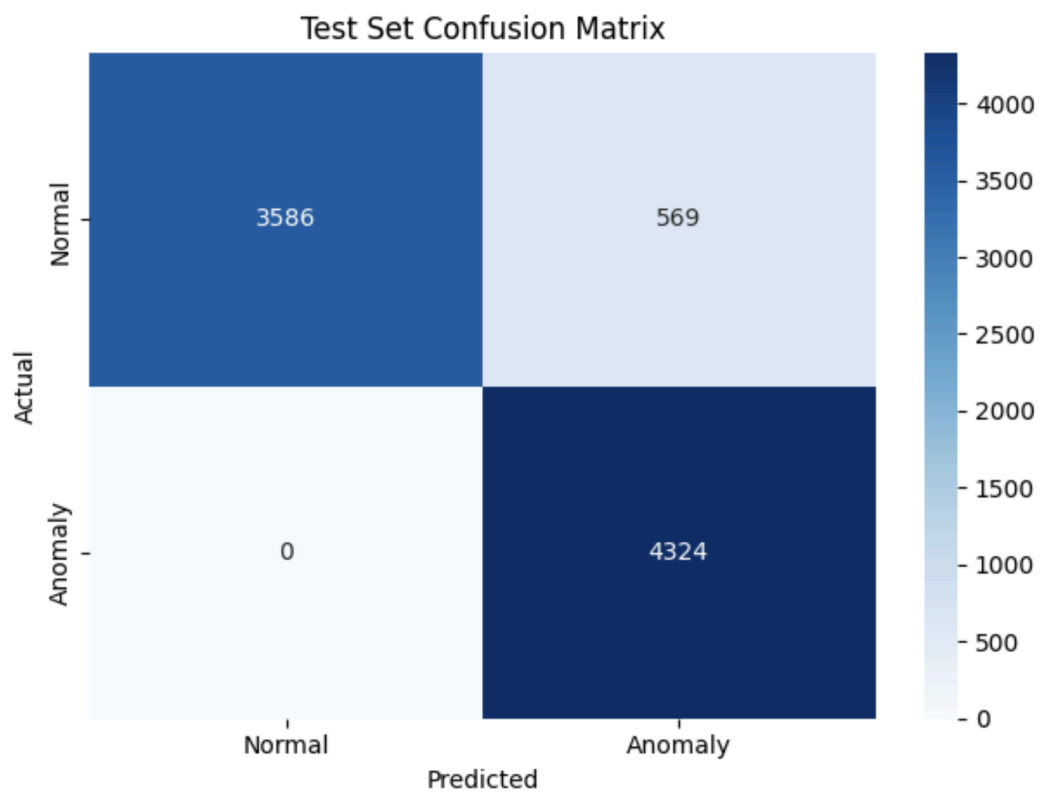
Part 2











Train set evaluation

	precision	recall	f1-score	support
Normal (0)	1.00	0.94	0.97	10478
Anomaly (1)	0.95	1.00	0.97	10872
accuracy			0.97	21350
macro avg	0.97	0.97	0.97	21350
weighted avg	0.97	0.97	0.97	21350

Test set evaluation

	precision	recall	f1-score	support
Normal (0)	1.00	0.86	0.93	4155
Anomaly (1)	0.88	1.00	0.94	4324
accuracy			0.93	8479
macro avg	0.94	0.93	0.93	8479
weighted avg	0.94	0.93	0.93	8479

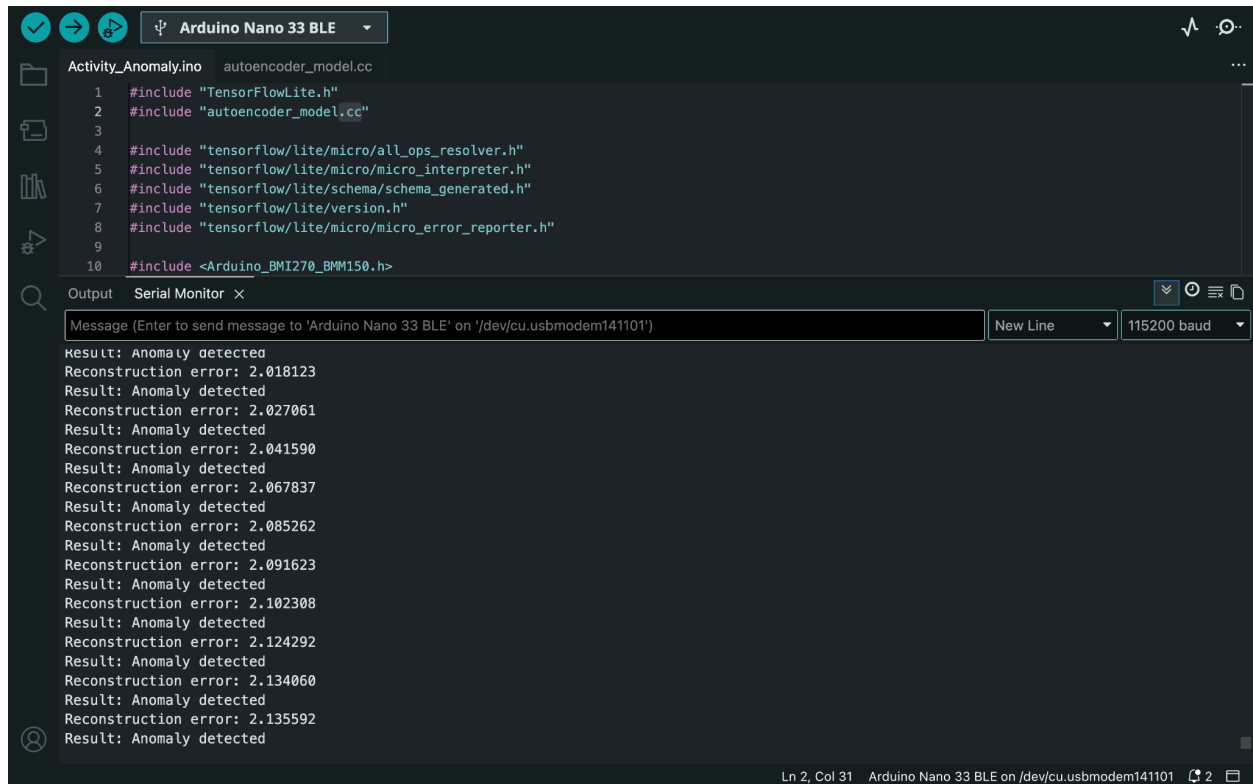
The screenshot shows the Arduino IDE interface. The top toolbar includes icons for running, uploading, and checking the code. The file explorer on the left shows the project structure. The main editor displays the code for `Activity_Anomaly.ino`, which includes headers for TensorFlow Lite and an autoencoder model. The Serial Monitor at the bottom shows the output of the program, which consists of a series of "Result: Normal activity" and "Reconstruction error: [value]" messages. The status bar at the bottom indicates the current line and column in the code.

```

Activity_Anomaly.ino  autoencoder_model.cc
1  #include "TensorFlowLite.h"
2  #include "autoencoder_model.cc"
3
4  #include "tensorflow/lite/micro/all_ops_resolver.h"
5  #include "tensorflow/lite/micro/micro_interpreter.h"
6  #include "tensorflow/lite/schema/schema_generated.h"
7  #include "tensorflow/lite/version.h"
8  #include "tensorflow/lite/micro/micro_error_reporter.h"
9
10 #include <Arduino_BMI270_BMM150.h>

Output  Serial Monitor  X
Message (Enter to send message to 'Arduino Nano 33 BLE' on '/dev/cu.usbmodem141101')  New Line  115200 baud

Result: Normal activity
Reconstruction error: 1.423664
Result: Normal activity
Reconstruction error: 1.374877
Result: Normal activity
Reconstruction error: 1.365701
Result: Normal activity
Reconstruction error: 1.370466
Result: Normal activity
Reconstruction error: 1.323678
Result: Normal activity
Reconstruction error: 1.264898
Result: Normal activity
Reconstruction error: 1.211544
Result: Normal activity
Reconstruction error: 1.203536
Result: Normal activity
Reconstruction error: 1.201162
Result: Normal activity
Reconstruction error: 1.132491
Result: Normal activity
  
```



The screenshot shows the Arduino IDE interface. The top toolbar includes icons for checking, uploading, and sending data, along with a dropdown menu for the board, currently set to 'Arduino Nano 33 BLE'. The left sidebar shows a file explorer with 'Activity_Anomaly.ino' selected. The main editor displays the code for 'autoencoder_model.cc', which includes TensorFlow Lite headers and an Arduino BMI270 sensor header. The Serial Monitor at the bottom shows a series of messages: 'Result: Anomaly detected' followed by 'Reconstruction error: [value]', where the values are approximately 2.018123, 2.027061, 2.041590, 2.067837, 2.085262, 2.091623, 2.102308, 2.124292, 2.134060, and 2.135592. The Serial Monitor settings are set to 'New Line' and '115200 baud'.

```
1 #include "TensorFlowLite.h"
2 #include "autoencoder_model.cc"
3
4 #include "tensorflow/lite/micro/all_ops_resolver.h"
5 #include "tensorflow/lite/micro/micro_interpreter.h"
6 #include "tensorflow/lite/schema/schema_generated.h"
7 #include "tensorflow/lite/version.h"
8 #include "tensorflow/lite/micro/micro_error_reporter.h"
9
10 #include <Arduino_BMI270_BMM150.h>
```

Result: Anomaly detected
Reconstruction error: 2.018123
Result: Anomaly detected
Reconstruction error: 2.027061
Result: Anomaly detected
Reconstruction error: 2.041590
Result: Anomaly detected
Reconstruction error: 2.067837
Result: Anomaly detected
Reconstruction error: 2.085262
Result: Anomaly detected
Reconstruction error: 2.091623
Result: Anomaly detected
Reconstruction error: 2.102308
Result: Anomaly detected
Reconstruction error: 2.124292
Result: Anomaly detected
Reconstruction error: 2.134060
Result: Anomaly detected
Reconstruction error: 2.135592
Result: Anomaly detected

What threshold did you choose for anomaly detection, and why?

A threshold of 0.5 was chosen as a starting point between the normal and anomaly error distributions. It can be adjusted up to reduce false positives or down to catch more anomalies after inspecting the histogram.

How does reconstruction error help distinguish normal and anomalous activity?

The autoencoder is trained only on normal data, so it learns to reconstruct normal patterns well with low error. Anomalous windows have patterns the model never saw, so it reconstructs them poorly and produces high error.

What information from the Python notebook must be preserved when deploying to Arduino?

The reconstruction error threshold must be copied into the Arduino sketch exactly, since it is the decision boundary. The input dimension (300) and the int8 quantization scale/zero-point parameters are also baked into the model and must match.

Why is it important for the Arduino sketch to use the same preprocessing assumptions as the notebook?

The model was trained on raw accelerometer values from a specific sensor placement, so the Arduino must read the same axes in the same order. If the input format differs, the reconstruction error values will be meaningless and the threshold will not work correctly.

What are the main limitations of this anomaly detection method when deployed on a microcontroller?

The model only knows what "normal" looked like during training, so it can fail if normal behavior drifts over time or varies between users. Memory and compute constraints on the Arduino also limit model size, which reduces the complexity of patterns the autoencoder can learn.